Orientações para configuração do ambiente de trabalho do app ReMap

Autor: José Carlos Soares Junior

Documentação de orientações sobre como instalar e configurar o ambiente de trabalho para o app ReMap. Aqui você irá encontrar informações desde a associação do projeto do GitHub até as configurações de desenvolvimento, incluindo erros apresentados em algum momento no passado, e como resolvê-los.

Fluxo Default (seguir esses passos a princípio, pois idealmente deveria funcionar)

Visão Geral

Quatro etapas principais:

1. **Clonar o repositório** via Git (usando RStudio ou terminal);

2. **Acessar diretamente o projeto** via arquivo `.Rproj` clonado para ter o ambiente Golem configurado automaticamente;

3. **Regenerar a infraestrutura Golem**, caso o `.Rproj` não esteja presente, recriar a estrutura de pacote sem modificar o remoto GitHub. Além disso, verificar e corrigir a consistência do pacote;

4. **Passos para desenvolvimento** após o pacote ter sido devidamente estruturado.

---

1. Pré-requisitos

1. **Git** instalado no sistema (Windows, macOS ou Linux).

2. **R** (>= 4.0) e **RStudio** instalados.

3. Pacote **golem** instalado via CRAN:

```
install.packages("golem")
```

4. Pacotes auxiliares:

```
install.packages(c("devtools", "remotes", "usethis"))
```

---

2. Clonando o repositório

2.1 Via RStudio

1. No RStudio, vá em **File** → **New Project…**.
2. Selecione **Version Control** e depois **Git**.
3. Cole a URL HTTPS do seu repositório (por exemplo, `https://github.com/usuario/meu-golem-app.git`).
4. Escolha a pasta local onde deseja clonar e confirme.

2.2 Via Terminal

```
git clone https://github.com/usuario/meu-golem-app.git
cd meu-golem-app
```

---

3. Preparando o projeto Golem

1. **Abrir o `.Rproj`** que já existe no repositório:
   - No RStudio, selecione **File** → **Open Project…** e aponte ao arquivo `.Rproj`.
2. **Caso o `.Rproj` não esteja** (por exemplo, se não tiver sido versionado):
   `golem::create_golem(path = ".", with_git = FALSE)`

Isso gera o esqueleto completo de um pacote Golem sem recriar o repositório Git.

3. **Executar o script inicial** para ajustar `DESCRIPTION`, configurar `golem-config.yml` e criar infraestrutura de testes:

   ```
   source("dev/01_start.R")
   ```

4. **Instalar dependências** declaradas no `DESCRIPTION`:

   ```
   remotes::install_deps(dependencies = TRUE)
   ```

5. **Verificar** a consistência do pacote:

   ```
   devtools::check()
   ```

---

4. Executando e desenvolvendo localmente

1. Carregue todo o pacote em modo de desenvolvimento:

   ```
   devtools::load_all()
   ```

2. Rode o ambiente de desenvolvimento do Golem (hot-reload):

   ```
   dev/02_dev.R
   ```

   0u

   ```
   source("dev/run_dev.R")
   ```

3. Para lançar a aplicação como pacote:

   ```
   shinyremap::run_app()
   ```

4. Faça alterações no diretório `R/`, use `golem::add_module()`, `golem::use_utils_ui()`, etc., e teste novamente com `run_app()`.

---

5. Integração com GitHub (usethis) caso necessário

Se o repositório local perdeu a referência `origin`, ou se você recriou o esqueleto Golem:

1. Inicialize Git (se necessário):

   ```
   usethis::use_git()
   ```

2. Configure o remoto e faça o push inicial:

   ```
   usethis::use_github(protocol = "https")
   ```

3. A partir daí, use `git push`/`pull` normalmente.

---

6. Dicas Adicionais

- **Branching**: crie branches específicas para novas funcionalidades ou correções com `git checkout -b feature/nova-coisa` (ou diretamente via menus da interface RStudio e GitHub).
- **Documentação**: use `golem::document_and_reload()` para gerar roxygen e atualizar `NAMESPACE`.
- **Testes**: ative infraestrutura de testes com `golem::use_recommended_tests()`.
- **CI/CD**: adicione badge e workflow via `usethis::use_github_action()` caso achar necessário.
- **Monitoramento**: use logs via `req_handlers()` em `golem` e visualize com `shinyValidator`.

---

7. Resumo de funções frequentemente utilizadas no fluxo de trabalho:

devtools::document() **atualiza o namespace com seu roxygen**

devtools::load_all() **recarrega o pacote**

shinyremap::run_app() **roda localmente o app (caso o pacote se chamar shinyremap)**

renv::install("username/reponame") **resolve renv erro instalando package do proprio rep do GitHub**

renv::update("xfun") **atualiza pacote xfunc no ambiente**

golem::add_shinyappsio_file() **deploy/republish**

Problemas e soluções

Executar o script de start (referente ao passo 3.3)

```
source("dev/01_start.R")
```

Saída típica e soluções:

- **Instalação automática de pacotes** (`attachment`, `dockerfiler`, `renv`, `rsconnect`, etc.)
    - Cada pacote é requerido para geração de namespace, contêiner Docker, lock de dependências e deploy
- **Erro de Rtools ausente**

  ```
  WARNING: Rtools is required to build R packages…
  ```

    - **Solução:** Instalar Rtools 4.4 e adicioná-lo ao PATH do Windows (veja seção 3)
- **Erro de pacote faltando** (`there is no package called 'openxlsx'`)
    - **Solução:**

      ```
      install.packages("openxlsx", dependencies=TRUE)
      ```

- *Por que?* Necessário para leitura/escrita de arquivos Excel dentro do app

---

Instalar e configurar Rtools e ferramentas de build (geralmente referente ao passo 3.5, caso necessário)

Caso esteja configurando um ambiente R recém instalado, pode ser necessário seguir os passos abaixo (ou algum deles). Lembre-se que o próprio R pode apresentar opções via UI para resolver a falta de dependências (o R algumas vezes apresenta uma janela popup para instalação do Rtools sem precisar rodar os códigos diretamente).

1. **Baixar e instalar** Rtools 4.4 do CRAN:
   https://cran.r-project.org/bin/windows/Rtools/ cran.r-project.org

2. **Adicionar ao PATH**:

   - Manual: *System Properties* → *Advanced* → *Environment Variables* → *Path* e insira `C:\rtools44\usr\bin` antes de `R\bin`

   - Alternativa `.Renviron`:

     ```
     write('PATH="${RTOOLS40_HOME}\\usr\\bin;${PATH}"',
     file="~/.Renviron", append=TRUE)
     ``` :contentReference[oaicite:7]{index=7}

3. **Verificar ferramentas**:

   ```
   install.packages("pkgbuild")
   pkgbuild::check_build_tools(debug=TRUE)
   ```

   - Deve retornar: *"Your system is ready to build packages!"*

Ajustes no `.Rprofile` (geralmente referente ao passo 3.5, caso necessário)

Para garantir que o PATH seja carregado sempre:

1. **em** `~/.Rprofile:`

```
Sys.setenv(PATH = paste("C:/Rtools/bin;C:/rtools44/usr/bin", Sys.getenv("PATH"), sep=";"))
```

2. **Verificação de Integridade com** `devtools::check()`:

   ```
   devtools::check()
   ```

Outros Erros Críticos e Soluções (geralmente referente ao passo 3.5)

1. **Vignette dependency 'knitr' sem vignettes**

   - **Solução:** Remover `knitr` de `Suggests:` e campo `VignetteBuilder:` no **DESCRIPTION**

2. **Namespace dependencies missing** (`gt`, `openxlsx`, `tidyr`)

   - **Solução:** Incluir em `Imports:` do **DESCRIPTION**:

     ```
     Imports:
      gt,
      openxlsx,
      Tidyr
     ```

3. **Invalid file names** (`_disable_autoload.R`)

   - **Renomear ou excluir** arquivos que comecem com `_`, que não seguem convenções de R

Warnings e Notes (melhoria contínua)

- **Conflito de imports** (`bs4Dash::column` vs `shiny::column`)

  Essas mensagens ocorrem porque você está importando funções com o mesmo nome de dois pacotes (`bs4Dash` e `shiny`).

Na seção **Imports** do seu `NAMESPACE`, importe apenas as funções que você efetivamente usa, por exemplo:

- Refine `NAMESPACE` com `importFrom(bs4Dash, box, dashboardPage)` e `importFrom(shiny, column, fluidRow)`

Isso elimina o conflito de nomes e esclarece quais funções vêm de cada pacote.

- **Pacotes pesados em Imports**

  Ex: Imports includes 23 non-default packages. … Move as many as possible to Suggests

  - **Sugestão:** pacotes usados apenas em módulos específicos (por exemplo, `highcharter`, `gtsummary` ou `readxl`) podem ir para **Suggests** e serem carregados condicionalmente via `requireNamespace()` dentro da função que os utiliza.

- **Arquivo de tamanho grande**

  Ex: installed size is 63.0Mb … sub-directory ‘app’ 62.8Mb

  - **Sugestão:** se possível, mova datasets ou assets muito grandes para um repositório de dados separado ou reduza o tamanho de arquivos em `inst/app/data/`.

- **Arquivo não-ASCII em R/**

  Ex: Found the following files with non-ASCII characters: …

  - **Sugestão:** execute:

    tools::showNonASCIIfile("R/app_ui.R")

    e substitua caracteres (acentos) por escapes Unicode `\u00e3`, `\u00e9` etc., ou remova acentuação em nomes de objetos.